

Hybrid Feature-Based Detection of AI-Generated Text

Finn Fonteijn

s2899078

f.f.fonteijn@student.utwente.nl

Linn Gregussen

s3422372

l.y.gregussen@student.utwente.nl

Abstract

1 Introduction

The availability of large language models (LLMs) creates a demand for reliable detection of AI-generated text in domains such as academia and journalism. Recent surveys indicate that hybrid approaches to AI detection, combining statistical and neural methods, often outperform single-modality detectors in both robustness and generalization (Wu et al., 2025; Su and Wu, 2024). A recent study by Erol et al. (2025) exemplifies these challenges in academic contexts, evaluating three commercial AI detectors on 250 human-written neurosurgery abstracts from pre-ChatGPT journals and 750 AI-generated versions (using ChatGPT 3.5, 4, and 4o). While the detectors achieved high AUC scores (0.96–1.00), they misclassified up to 30.4% of human texts as AI-generated, raising ethical concerns about false accusations in peer review.

Motivated by such findings, this project takes a dual approach to AI text detection. First, we conduct a feature-based analysis to identify which linguistic and statistical patterns are most discriminative. Second, we implement an ensemble-based detector combining multiple classification approaches to explore whether such combinations can improve practical deployment robustness.

This project addresses two research questions:

RQ1: *Which linguistic or statistical features are most effective in distinguishing human- vs. AI-generated text?*

RQ2: *How does ensemble-based classification improve robustness compared to individual classifiers?*

2 Related Work

Wu et al. (2025) conducted a comprehensive review of the state-of-the-art in LLM-generated text detection in 2025. They classify LLM-generated text detectors into three categories: watermarking

technology where the generator includes identifiable patterns in the text to regulate and detect its origin; statistics-based detectors which analyze inherent patterns of the generated text; and neural-based detectors which use machine learning models to classify the generated text. The detectors in the survey report performance using different metrics, but among the best performing are the two statistics-based detectors "Ghostbuster" with average F1 score of 99.0 and GECScore with an AUROC of 98.62%. Among the neural-based detectors, DNA-GPT achieved "nearly 100%" detection accuracy in distinguishing between human-written and GPT-generated texts on multiple datasets. The watermarking methods are noted for their robustness against adversarial attacks, e.g. reliable classification when up to 50% of the tokens were modified, and low false-positive rates, making them a promising approach for reliable detection.

However, robustness against adversarial attacks remains a significant challenge. Weber-Wulff et al. (2023) examine the reliability of AI text detection tools in academic settings and found that obfuscation techniques, e.g. manual editing and machine paraphrasing, significantly reduce the accuracy of detection tools, with approximately 50% of the texts that had undergone some type of post-generation editing being misclassified as human-written. Similar results, with detectors being less accurate when subjected to adversarial attacks, are reported by Wu et al. (2025) and Pudasaini et al. (2025).

Multiple surveys indicate that hybrid approaches, combining statistical and neural methods, often outperform single-modality detectors in robustness and generalization (Wu et al., 2025; Su and Wu, 2024).

3 Data

To ensure our model generalizes across diverse English language sources, domains, and LLMs, and to mitigate overfitting to any single dataset's biases, we utilize multiple datasets for training and

Dataset	Samples	Human (%)	AI (%)
HC3	85K	68.5	31.5
sunilthite	29K	60.1	39.9
DAIGT V2	45K	61.0	39.0
AH&AITD	12K	50.0	50.0

Table 1: Dataset size and class distributions

evaluation. This approach allows for robust testing of features and hybrids in varied contexts. The datasets include AH&AITD (Arslan’s Human and AI Text Database) with texts across domains including articles, abstracts, stories, news, and reviews (Akram, 2023); the LLM - Detect AI Generated Text Dataset (Kaggle sunilthite) containing essays (Thite, 2023); the DAIGT V2 Train Dataset (Kaggle) with essays (Deotte, 2023); and the Human ChatGPT Comparison Corpus (HC3) comprising Q&A from domains such as finance, medicine, law, and open-domain questions (Guo et al., 2023).

3.1 Dataset Balancing

The datasets showed varying degrees of class imbalance between the target classes human-written (0) and AI-generated (1), ranging from 31.5% AI-generated in the HC3 dataset to 50.0% AI-generated in the AH&AITD dataset. The dataset’s class distribution is shown in Table 1.

The dataset was balanced using a per-source strategy, where each source contributes an equal number of human-written and AI-generated samples according to its configured proportion of the total dataset size, and limited by the smallest class per source. This ensures balanced representation across both the source datasets and the target classes.

Building on this balanced dataset foundation, we now describe our methodological approach for AI text detection, which combines feature-based analysis with ensemble-based classification.

4 Method

The project methodology consists of two efforts to address different aspects of AI text detection: a feature-based analysis and an ensemble-based detector.

4.1 Approach 1: Feature-Based Analysis

The feature-based analysis focuses on understanding which linguistic and statistical features are most informative in classifying human- vs. AI-generated text. For each text in the dataset, a set of features

are extracted (section 4.1.1). These features are then used to train three lightweight classifiers: logistic regression and random forest using scikit-learn (Pedregosa et al., 2011), and XGBoost (Chen and Guestrin, 2016).

The dataset was balanced as described in section 3.1. Feature importance rankings were retrieved and compared across these three models, giving insights on which linguistic patterns are consistently important for distinguishing human- vs. AI-generated text.

4.1.1 Feature Extraction

For each entry in the dataset, the following features were extracted using the spaCy library (Honnibal et al., 2020) with the en_core_web_sm model for all linguistic processing tasks, including tokenization, lemmatization, part-of-speech tagging, and dependency parsing.

Lexical Features Four lexical features were chosen and extracted. Average word length and word length variance to measure vocabulary complexity. Root Type-Token Ratio (RTTR), and hapax legomena ratio to measure lexical diversity.

The Root Type-Token Ratio (RTTR) is a token-type ratio that accounts for document size by normalizing by the square of the document length (Reviriego et al., 2024). It was calculated by lemmatizing the text, counting the unique types and dividing by squared text length. A hapax legomenon refers to a word (or expression) occurring only once in a context, and here measures the proportion of words that appear exactly once in a sample.

Syntactic Features Sentence-level statistics were chosen to measure sentence complexity, and include average sentence length and variance. Parse tree depth features (average, maximum and variance) were extracted to measure syntactic complexity. Parse depth is computed as the maximum number of ancestors for any token in a sentence.

Punctuation Features The ratio of punctuation marks to word count was calculated for comma, period, exclamation mark, and question mark. Additionally, the overall punctuation ratio was included.

Part-of-Speech Features POS tags were extracted using spaCy’s coarse-grained POS tags (token.pos_) rather than fine-grained tags (token.tag_) to capture overall patterns in word class usage, fine grained tags were excluded as they may be too specific for the objective, and

yield sparse data. The ratio of each POS category (noun, verb, adjective, adverb, etc.) to total tokens was calculated as well as a noun-to-verb ratio as a derived feature.

Perplexity Perplexity is defined as the inverse test set probability according to the model used for calculating the perplexity score, normalized by the length of the text (Jurafsky and Martin, 2025). It is an evaluation of the model’s ability to predict uniformly among the set of tokens in a corpus, or in other words how "surprised" the model is by the text, where a lower perplexity indicates more predictable/less surprising text for the model. Perplexity was calculated using EleutherAI’s Pythia 1.4B model (Biderman et al., 2023), an open-source model trained on the Pile dataset (Gao et al., 2020). Pythia was specifically chosen for its reproducibility and transparency in training, providing duplicated data variants that ensure clean perplexity signals without contamination artifacts from memorized training data (Biderman et al., 2023). This makes it ideal for AI text detection where we need to distinguish true predictive patterns from artificial recognition of memorized content.

Burstiness Features In statistics, burstiness is a measure of the variability in data points over time. Burstiness captures the "clumpiness" in text structure, where AI text might show more uniform patterns while human writing exhibits greater variation. The core burstiness metric uses the Goh-Barabasi coefficient (Goh and Barabási, 2008): $B = \frac{\sigma - \mu}{\sigma + \mu}$, where σ is the standard deviation and μ is the mean, this measure is designed for analyzing variation in sequential data distributions, which is what we wish to analyze. The formula yields a burstiness score between -1 (uniform) and 1 (highly variable). For each text input, ten burstiness features were originally extracted, measuring variation at the sentence, word, and clause levels, including sentence length burstiness, positional variation (comparing first and second halves) and clause length burstiness. After correlation analysis, six burstiness features were retained.

4.1.2 Feature Selection

To reduce collinearity, a correlation analysis was performed on all the described extracted features, and from pairs with high pairwise correlation ($r > 0.8$), one feature was dropped. The feature to be dropped was selected based on whether it was correlated with more other features, and which feature

Table 2: List of selected/considered linguistic features

Category	Feature Name
Lexical	average_word_length
	word_length_variance
	rttr_root_type_token_ratio
	hapax_legomena_ratio
Syntactic	average_sentence_length
	sentence_length_variance
	average_parse_depth
	maximum_parse_depth
	parse_depth_variance
Punctuation	comma_ratio
	period_ratio
	exclamation_mark_ratio
	question_mark_ratio
	punctuation_per_word
Part-of-Speech (POS)	pos_ratio_<tag>
	noun_to_verb_ratio
Burstiness	sentence_length_burstiness
	sentence_length_iqr
	position_burstiness_diff
	long_sentence_ratio
	word_length_burstiness
	clause_length_burstiness
	sentence_length_cv [†]
	sentence_length_range_ratio [†]
	adjacent_sentence_variation [†]
	short_sentence_ratio [†]
Other	perplexity

[†] Removed; high correlation ($r > 0.8$) with other feature(s)

would be more informative in the project context. After this selection process, 40 linguistic and statistical features were left for model training. The list of selected and dropped features is shown in Table 2, for the extended list with all POS-tags, see Appendix A.

4.2 Approach 2: Ensemble-Based Detector

The ensemble-based detector combines five classifiers to build a practical, robust AI text detector. The classifier outputs are combined using two methods: majority voting and stacking.

4.2.1 Base Classifiers

The ensemble-based detector is made up of multiple text-based classifiers and the XGBoost classifier from Approach 1 (Section 4.1) leveraging different perspectives on the classification task.

BERT Classifier BERT captures contextual word representations through its bidirectional transformer architecture, letting it capture semantic patterns in text. For this project the BERT classifier available at Hugging Face as

google-bert/bert-base-uncased is used (Devlin et al., 2019).

DeBERTa Classifier A DeBERTa-based classifier using the microsoft/deberta-v3-base model. DeBERTa uses disentangled attention mechanisms and enhanced mask decoder, this way a word’s content can be separated from its position and processed independently, this gives it a clearer sense of what’s being said and where in the sentence it’s happening, potentially capturing more nuanced linguistic patterns for the AI classification task (He et al., 2023).

Naive Bayes Classifiers Two Naive Bayes classifiers were included in the ensemble, one using the full vocabulary and one with stopwords removed.

XGBoost Classifier The XGBoost classifier described in section 4.1 was also added to the ensemble, giving the ensemble access to the explicit linguistic features extracted from the text.

4.2.2 Ensemble Methods

Two ensemble methods were implemented: majority voting and stacking.

Majority Voting In the voting approach, each of the five base classifiers independently predicts the class label for a given text. The final prediction of the ensemble is then determined by a majority vote across the classifiers. This simple aggregation reduces individual model errors, increasing robustness against the misclassification of a single model.

Stacking In the stacking approach, the predictions of all base classifiers on the training set are used as features to train a final estimator model. Three models were tested for this final estimator: Logistic Regression, Random Forest, and XGBoost, all implemented using the scikit-learn library (Pedregosa et al., 2011). Like majority voting, individual errors are reduced, but the use of a final estimator allows the ensemble to learn optimal weights for the different classifiers, potentially improving the overall performance of the ensemble.

4.3 Evaluation

4.3.1 Classification Metrics

All classifiers are evaluated on a held-out test set using multiple performance metrics: accuracy, precision, recall, F1-score (macro, weighted, and per-class), and AUROC (Area Under the ROC Curve).

AUROC provides a threshold-independent measure of classifier performance across all classification thresholds (0-1, with higher values indicating better discrimination between classes), with random baseline performance at 0.5. For the ensemble, both individual model performance and complete ensemble performance are reported, to allow comparison between the internal ensemble methods and the overall ensemble performance.

4.3.2 Feature Importance Analysis

For the feature-based analysis, we extract and compare feature importances from each model: absolute coefficient values from logistic regression (normalized to sum to 1), Gini importance from random forest, and gain-based importance from XGBoost. By ranking features across models, we identify which linguistic patterns are consistently discriminative regardless of the classification algorithm.

SHAP Summary Plots SHAP (SHapley Additive exPlanations) values are calculated and visualized for the XGBoost classifier to provide interpretable explanations of feature contributions using the shap library (Lundberg, 2017). The SHAP summary plot visualizes the distribution and magnitude of each feature’s impact across all test predictions by the XGBoost model, showing which features are most influential for the model’s decisions, and in which direction.

5 Experiment and Results

5.1 Experiment Setup

The complete implementation of this project is available in the github repository nlp-ai-detector (Fontijn and Gregussen, 2025).

5.1.1 Dataset Configuration

The combined dataset consisted of 112,848 samples from the datasets described in Section 3 and was balanced per source and per class as described in Section 3.1. Table 3 shows the contribution of each source to the final combined dataset. The data was split into training (80%) and test (20%) sets using stratified sampling with a fixed seed to ensure reproducibility and maintain class balance across splits, giving 90,278 training samples and 22,570 test samples.

5.1.2 LLM Setup

The LLM-models were downloaded from Hugging Face, and run using the transformers library (Wolf

Dataset	Samples	Proportion (%)
HC3	43,000	38.1
DAIGT V2	34,994	31.0
sunlthite	23,274	20.6
AH&AITD	11,580	10.3
Total	112,848	100.0

Table 3: Combined dataset composition

et al., 2020). The models were configured with the following hyperparameters:

Feature-Based Classifiers Logistic Regression used default scikit-learn parameters with random state 42. Random Forest was configured with max depth 10 and random state 42. XGBoost used 100 estimators, max depth 6, learning rate 0.1, random state 42, and eval metric 'logloss'.

Transformer-Based Classifiers BERT (google-bert/bert-base-uncased) and DeBERTa (microsoft/deberta-v3-base) classifiers were configured identically: maximum sequence length 128 tokens, 3 training epochs, learning rate 0.005, batch size 64. For both models, pre-trained embeddings were frozen and only a linear classification head was trained.

Naive Bayes Classifiers Both Naive Bayes classifiers used alpha smoothing parameter 1.0 and bag-of-words features with CountVectorizer. The first used the full vocabulary, while the second had 'english' stopwords removed.

Ensemble Configuration The majority voting ensemble used unweighted voting across the five base classifiers. For stacking, all meta-classifiers Logistic Regression, Random Forest, and XGBoost, used library default parameters.

Perplexity Calculation Perplexity features were computed using the EleutherAI/pythia-1.4b model with a maximum sequence length of 1,536 tokens and a stride of 1,536 to avoid overlapping windows.

5.2 Results

5.2.1 Feature-Based Analysis

Classification Performance Table 4 presents the performance of the three feature-based classifiers on the test set (22,570 samples, 11,285 per class). XGBoost achieved the highest performance with 97.45% accuracy and 0.997 AUROC, followed by Random Forest (96.06% accuracy, 0.993 AUROC)

Metric	Logistic Regr.	Random Forest	XGBoost
Accuracy	0.928	0.961	0.975
AUROC	0.972	0.993	0.997
<i>Human-Written</i>			
Precision	0.931	0.949	0.969
Recall	0.925	0.973	0.980
F1-Score	0.928	0.961	0.975
<i>AI-Generated</i>			
Precision	0.926	0.973	0.980
Recall	0.931	0.948	0.969
F1-Score	0.928	0.960	0.974
F1 (Macro)	0.928	0.961	0.975
F1 (Weighted)	0.928	0.961	0.975

Table 4: Performance metrics for feature-based classifiers. Best values in bold.

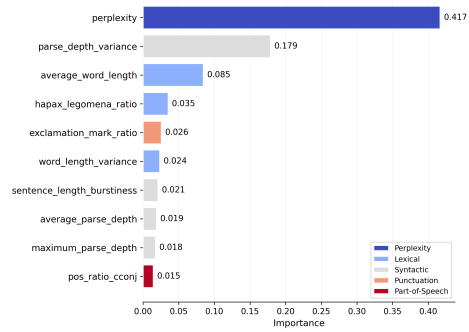


Figure 1: Top 10 feature importances for Logistic Regression classifier

and Logistic Regression (92.82% accuracy, 0.972 AUROC).

Feature Importance Analysis Figures 1, 2, and 3 show the top 10 features for each classifier. Perplexity dominates across all models (importance 0.334-0.430), followed by lexical features (average word length, hapax legomena ratio). The models show differences in which type of features are more discriminative: Logistic Regression and Random Forest emphasize syntactic complexity (four syntactic features each), while XGBoost relies more heavily on POS ratios (four POS features vs. one for LR and none for RF).

SHAP Summary Plot Figure 4 shows the SHAP analysis for the XGBoost classifier. Perplexity is the most discriminative feature, followed by lexical features (hapax legomena ratio, average word length, RTTR). High perplexity values (red) are strongly associated with human-written text, while low perplexity (blue) is strongly associated with AI-generated text. For the lexical features, lower values are associated with human-written text, that is less unique words, shorter words and a smaller

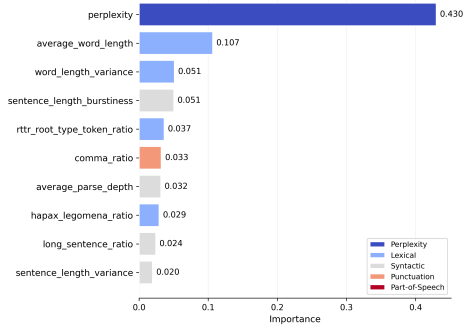


Figure 2: Top 10 feature importances for Random Forest classifier

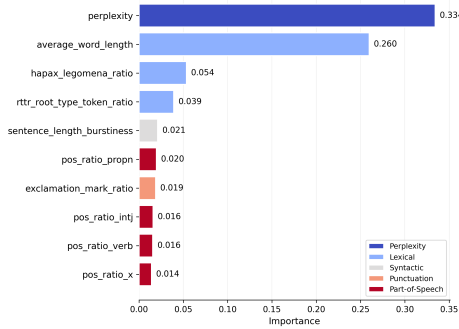


Figure 3: Top 10 feature importances for XGBoost classifier

range of words is associated with human-written text.

5.2.2 Ensemble-Based Detector

Classification Performance Table 5 presents the performance of all classifiers in the ensemble-based detector. The feature-based XGBoost classifier achieved the highest performance (97.5% accuracy, 0.997 AUROC). Among the text-based classifiers, DeBERTa performed best (94.2% accuracy, 0.988 AUROC), followed by BERT (92.1% accuracy, 0.975 AUROC).

Ensemble Performance The ensemble methods demonstrated robust performance across multiple evaluation metrics, with voting and averaging approaches providing consistent improvements over individual classifiers in terms of stability and generalization.

Having examined both feature-based results and ensemble performance, we now discuss the key findings and their implications for AI text detection.

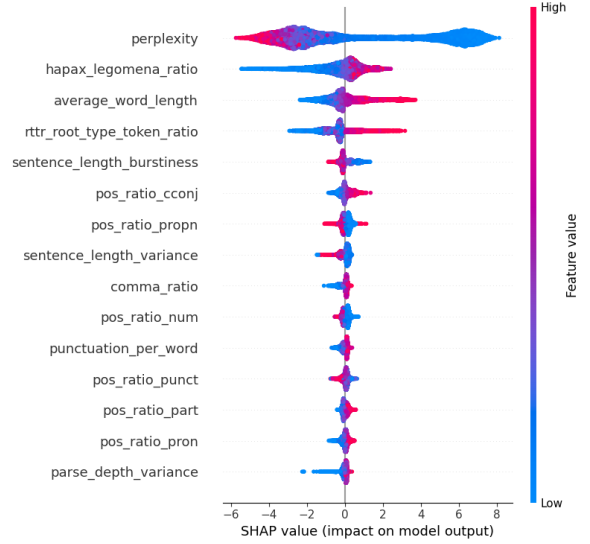


Figure 4: SHAP summary plot for XGBoost classifier. Positive values indicate AI-generated

Classifier	Accuracy	Macro F1	AUROC
Random Classifier	0.494	0.494	0.500
Naive Bayes	0.848	0.848	0.918
NB (No Stopwords)	0.852	0.851	0.920
BERT	0.921	0.921	0.975
DeBERTa	0.942	0.942	0.988
XGBoost	0.975	0.975	0.997

Table 5: Performance comparison of ensemble component models. Best values in bold.

6 Discussion and Conclusion

6.1 Feature Importance Findings

The consistency of perplexity as the top-ranked feature across all three models (importance scores: 0.334-0.430) indicates that perplexity captures fundamental differences in text generation patterns. As expected, the direction of this relationship is that a higher perplexity score is indicative of human-written text, i.e. that AI text is more predictable to the Pythia 1.4B model, likely because there are similarities in the training data of different LLMs.

Lexical features are also consistently among the top discriminators across all models. AI-generated text is characterized by longer words and greater vocabulary diversity. This shows that modern LLMs employ a more varied vocabulary than human writers, possibly reflecting their large and diverse training corpora, where the average human vocabulary reaches natural limits and domain constraints.

Apart from perplexity, the models show differences in the feature importance rankings. Logistic Regression and Random Forest emphasize syntac-

tic complexity features (parse depth variance, sentence structure), while XGBoost relies on simpler POS-based features. XGBoost achieves the highest performance (97.45% accuracy) while using arguably simpler features rather than more computationally expensive and complex syntactic parsing-based features. This suggests XGBoost’s gradient boosting effectively learns relationships between basic ratio-features that compensate for syntactic information. The specific POS features it emphasizes, ratios of proper nouns, verbs, interjections, and unusual tokens, may represent basic stylistic differences between human and AI-generated text.

Building on these feature-level insights, we now examine how ensemble-based approaches compare to individual classifiers in terms of detection performance.

6.2 Ensemble-Based Detector vs. Individual Classifiers

6.3 Method Limitations

A limitation of this project is that our datasets include only samples that are either entirely human-written or entirely AI-generated, with no mixed or (adversarially) modified samples. In real-world scenarios, users may use different techniques to fool the detectors and evade detection. Our models were not trained or evaluated on such samples and their robustness to these scenarios would require further evaluation before practical deployment.

The perplexity feature, which shows a high degree of discriminative power in our experiments, is calculated using a single language model, Pythia 1.4B trained on the Pile dataset, and may not generalize to text generated by models trained on substantially different data distributions.

The generalizability of our models is also limited by the fact that we do not test LLM-models beyond those represented in our datasets, which do not include the latest models, such as GPT-5.

6.4 Ethical and Societal Considerations

This project develops an AI detection tool, which can be used to detect AI-generated text in a range of domains such as academia, journalism or creative writing. Our intention is that knowledge about whether a text is AI-generated or not can be used in a "positive" way for society in general. In the mentioned domains, this could help prevent fraud in academic settings, or help journalists flag sources that may be AI-generated and require further inves-

tigation. Our tool explicitly tackles the problem of adversarial attacks against AI-detection tools by using an ensemble of classifiers where one will have to fool multiple models to successfully fool the tool. However, it is important to reflect upon how such a tool can be used to improve and create adversarial attacks/tools, both against this specific tool, but also against AI-detectors in general, by training an adversarial model by using this tool in a detector-evasion setup.

Another ethical consideration is that this tool does not have zero false positive or false negative rates. Incorrect classification of human-written text can have serious consequences for the individual, such as a student being accused of cheating, possibly leading to failing a course or even being expelled from a university. Incorrect classification of AI-generated text on the other hand can lead to spread of misinformation such as "fake" news or made up scientific findings with a "stamp" of authenticity. To minimize these potential harms, a tool such as this should be deployed with transparent confidence scores and clear disclaimers and explanations about its limitations.

6.5 Conclusion

This project investigated which linguistic and statistical features are most effective for AI-text detection (RQ1) and whether ensemble-based classification improves robustness (RQ2). Regarding RQ1, perplexity is identified as the most dominant discriminative feature across all models, with higher perplexity indicating human-written text. Lexical features, particularly average word length, hapax legomena ratio, and RTTR, consistently ranked among the top discriminators, indicating that AI-generated text has greater vocabulary diversity and longer words than human writing. The XGBoost classifier reached the highest performance (97.45% accuracy, 0.997 AUROC), demonstrating that explicit linguistic features can be an effective signal for AI text detection.

Regarding RQ2, we were unable to demonstrate clear improvements from the ensemble approach, as the XGBoost classifier with its overall high performance dominated the ensemble’s predictions. Ensemble methods are theoretically most beneficial in scenarios of adversarial attacks and domain shifts, but we have not performed such testing and cannot evaluate whether they improve detector robustness in these contexts. Future work should assess whether the added complexity provides

meaningful benefits over a single well-performing feature-based classifier.

References

- Arslan Akram. 2023. [AH&AITD: Arslan’s Human and AI Text Database](#). Dataset. Associated with the article: An Empirical Study of AI-Generated Text Detection Tools. *Advances in Machine Learning & Artificial Intelligence*, 4(2), 44–55.
- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. 2023. Pythia: a suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org.
- Tianqi Chen and Carlos Guestrin. 2016. [Xgboost: A scalable tree boosting system](#). In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’16*, page 785–794, New York, NY, USA. Association for Computing Machinery.
- Chris Deotte. 2023. [Daigt v2 train dataset](#). Dataset containing 44k human-written essays and AI-generated ones from different LLMs, including OSS ones.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). *Preprint*, arXiv:1810.04805.
- G. Erol, A. Ergen, B. Gülşen Erol, Ş. Kaya Ergen, T. S. Bora, A. D. Çölgeçen, B. Araz, C. Şahin, G. Bostancı, İ. Kılıç, Z. B. Macit, U. T. Sevgi, and A. Güngör. 2025. [Can we trust academic ai detective? accuracy and limitations of ai-output detectors](#). *Acta neurochirurgica*, 167(1):214.
- Finn Fonteiijn and Linn Gregussen. 2025. [NLP AI Detector](#).
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.
- K.-I. Goh and A.-L. Barabási. 2008. [Burstiness and memory in complex systems](#). *Europhysics Letters*, 81(4):48002.
- Biyang Guo, Xin Zhang, Ziyuan Wang, Minqi Jiang, Jinran Nie, Yuxuan Ding, Jianwei Yue, and Yupeng Wu. 2023. How close is chatgpt to human experts? comparison corpus, evaluation, and detection. *arXiv preprint arxiv:2301.07597*.
- Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. [Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing](#). *Preprint*, arXiv:2111.09543.
- Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. [spaCy: Industrial-strength Natural Language Processing in Python](#). Zenodo.
- Daniel Jurafsky and James H. Martin. 2025. [Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models](#), 3rd edition. Draft. Online manuscript released August 24, 2025.
- Scott M. Lundberg. 2017. Shap (shapley additive explanations). <https://github.com/shap/shap>. Accessed: 2025-06.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Shushanta Pudasaini, Luis Miralles-Pechuán, David Lillis, and Marisa Llorens Salvador. 2025. [Survey on AI-Generated Plagiarism Detection: The Impact of Large Language Models on Academic Integrity](#). *Journal of Academic Ethics*, 23(3):1137–1170.
- Pedro Reviriego, Javier Conde, Elena Merino-Gómez, Gonzalo Martínez, and José Alberto Hernández. 2024. [Playing with words: Comparing the vocabulary and lexical diversity of chatgpt and humans](#). *Machine Learning with Applications*, 18:100602.
- Yongye Su and Yuqing Wu. 2024. [Robust detection of llm-generated text: A comparative analysis](#). *Preprint*, arXiv:2411.06248.
- Sunil Thite. 2023. [Llm - detect ai generated text dataset](#). Dataset containing more than 28,000 essays written by students and AI-generated.
- Debora Weber-Wulff, Alla Anohina-Naumeca, Sonja Bjelobaba, Tomáš Foltýnek, Jean Guerrero-Dib, Oluamide Popoola, Petr Šigut, and Lorna Waddington. 2023. [Testing of detection tools for AI-generated text](#). *International Journal for Educational Integrity*, 19(1):26.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. [Transformers: State-of-the-art natural language processing](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Lidia Sam Chao, and Derek Fai Wong. 2025. [A survey on llm-generated text detection: Necessity, methods, and future directions](#). *Computational Linguistics*, 51(1):275–338.

Appendix

A Full List of Extracted Features

average_word_length, word_length_variance,
 rttr_root_type_token_ratio,
 hapax_legomena_ratio, average_sentence_length,
 sentence_length_variance, average_parse_depth,
 maximum_parse_depth, parse_depth_variance,

comma_ratio, period_ratio,
 exclamation_mark_ratio, question_mark_ratio,
 punctuation_per_word, pos_ratio_adj,
 pos_ratio_adp, pos_ratio_adv, pos_ratio_aux,
 pos_ratio_conj, pos_ratio_cconj, pos_ratio_det,
 pos_ratio_intj, pos_ratio_noun, pos_ratio_num,
 pos_ratio_part, pos_ratio_pron, pos_ratio_propn,
 pos_ratio_punct, pos_ratio_sconj, pos_ratio_sym,
 pos_ratio_verb, pos_ratio_x, noun_to_verb_ratio,
 sentence_length_burstiness, sentence_length_iqr,
 position_burstiness_diff, long_sentence_ratio,
 word_length_burstiness, clause_length_burstiness,
 perplexity